

MODUL 7

(SEARCHING)



DESKRIPSI TEMA

Searching merupakan suatu fungsi pencari, yang dirancang untuk mencari suatu data dari sekumpulan data. Pencarian data dapat dicari dalam kelompok data yang terurut (*sorted*) maupun tidak terurut (*unsorted*).

Pada minggu ini kita akan mempelajari beberapa algoritma untuk searching, yaitu *Sequential search*, *Binary search*, dan *Interpolation search* yang dikombinasikan dengan bahan pembelajaran pada minggu sebelumnya: *sorting*.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan berbagai algoritma searching (C₃).

PENUNJANG PRAKTIKUM

1. Software CodeBlocks

LANGKAH-LANGKAH PRAKTIKUM

Tutorial 1 – Sequential Search

Sequential search atau disebut juga sebagai *linear search* merupakan algoritma untuk pencarian data yang dilakukan secara berurutan dari depan (data awal) hingga ke belakang (data akhir). Pada *Sequential search* data tidak perlu diurutkan terlebih dahulu.

Cara pengaplikasian *sequential search* pada array adalah seperti berikut :



```
1  #include<stdio.h>
2  #include<stdbool.h>
3  #include<string.h>
4
5  int main(int argc, char **argv){
6      char data[10][50]={"sapi", "kambing", "guguk", "domba", "ayam", "bebek", "burung", "ikan", "lebah",
7      "kuda"};
8      int i;
9      bool check = false;
10     char key[20];
11     scanf("%s", key);
12     for(i = 0; i < 10; i++){
13         if(strcmpi(data[i], key) == 0){
14             check = true;
15             break;
16         }
17     }
18     if(check == true){
19         printf("Data Found !\n");
20         printf("%s", data[i]);
21     }
22     else{
23         printf("Data not Found !");
24     }
25     return 0;
26 }
27
```

Tutorial :

Bagaimana *best case* (case terbaik), *worst case* (case terburuk), dan *average case* (case rata rata) pada algoritma *sequential search*? jawablah berdasarkan apa yang anda pahami selama melakukan penyalinan potongan code diatas dalam suatu file txt dengan format nama **[NIM]_[Nama]_To1.txt!**

Tutorial 2 – Binary Search

Binary search merupakan algoritma untuk pencarian data yang memerlukan data yang telah di-sort terlebih dahulu, dilakukan secara berulang-ulang dengan membagi *search interval* menjadi 2 bagian satu per satu.

Langkah-langkah dalam melakukan *binary search* :

1. Urutkan data terlebih dahulu secara *ascending*.
(Binary search dapat dilakukan pada data yang diurutkan secara *descending*, namun untuk tutorial ini kita menggunakan cara *ascending* saja)
2. Membuat variabel seperti *low*, *mid*, dan *high* sebagai penanda / pembatas suatu data dan meng-input data yang ingin dicari yang akan dijadikan sebagai *key*.
3. Melakukan perulangan yang terdiri dari beberapa proses:
 - Menghitung nilai *mid*. Dimana $mid = (low + high) / 2$

- Membandingkan *key* dengan data yang ada di index *mid*, terdapat 3 kondisi, yaitu:
 - i. Jika sama maka proses selesai
 - ii. Jika *key* lebih kecil, maka nilai *high* berubah menjadi *mid-1*
 - iii. Jika *key* lebih besar, maka nilai *low* berubah menjadi *mid+1*
- 4. Kembali ke proses ketiga hingga nilai *low* lebih besar dari *high* atau *key* telah cocok dengan data yang ada pada *mid*.

Tutorial 3 – Interpolation Search

Interpolation Search merupakan algoritma untuk pencarian data yang dilakukan pada sekumpulan data dan merupakan pengembangan dari *binary search*.

Apabila *Binary Search* selalu memulai pengecekan pada index tengah, *Interpolation Search* akan memulai pengecekan di lokasi index yang berbeda, tergantung dengan nilai *key* yang akan di search. Adapun rumus yang digunakan dalam tiap iterasi pencarian yaitu :

$$\text{Posisi Relatif} = \left\{ \frac{\text{kunci} - \text{data}[\text{low}]}{\text{data}[\text{high}] - \text{data}[\text{low}]} \times (\text{high} - \text{low}) \right\} + \text{low}$$

Berikut merupakan salah satu contoh function dari *Interpolation Search*

```
int SearchInterpolation(int data[],int n,int searchKey){
    int position,low,high;
    low = 0;
    high = n-1;
    do{
        position = (searchKey - data[low]) * (high-low) / (data[high] - data[low]) + low;
        if(data[position] == searchKey) return position;
        if(data[position] > searchKey) high = position - 1;
        else low = position + 1;
    }while(searchKey >= data[low] && searchKey <= data[high]);
    return -1;
}
```

Tutorial

Telusurilah pencarian data dengan nilai 14261 data di bawah ini dengan metode Interpolation Search. Tulis nilai *low*, *high* dan perhitungan pencarian *position* di setiap iterasi hingga data ditemukan. Kerjakan dalam suatu file txt dengan format nama [NIM]_[Nama]_To2.txt!

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

13728	13613	16358	13610	14026	14261	14152	13411	17942
-------	-------	-------	-------	-------	-------	-------	-------	-------

Challenge 1

Perhatikan potongan code berikut !

```
#include<stdio.h>
#include<stdbool.h>
#include<string.h>

int main(int argc,char **argv){
    char data[10][50]={"sapi","kambing","guguk","domba","ayam","bebek","burung","ikan","lebah",
    "kuda"};
    int i;
    bool check = false;
    int jumlahdata = 10;
    int low = 0;
    int high = jumlah data -1;

    while(low<=high){
        int mid;
        if(strcmpi(data[i],mid)==0){
            check=true;
            break;
        }
    }

    if(check==true){
        printf("Data Found !\n");
        printf("%s",data[i]);
    }
    else{
        printf("Data not Found !");
    }
    return 0;
}
```

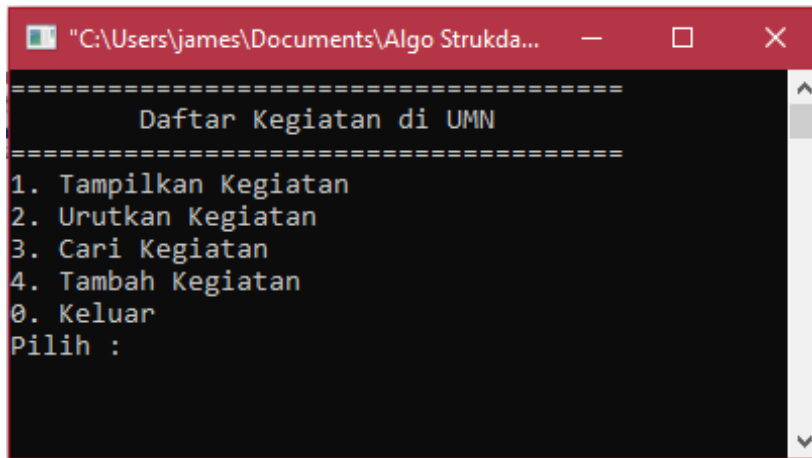
Jika kalian mengikuti potongan codingan binary search tersebut, adapun beberapa kesalahan. Perbaiki lah beberapa hal yang kurang dari Langkah melakukan binary search. Kerjakan dalam suatu file .c dengan format nama [NIM]_[Nama]_Challenge01.c

Challenge 2

Pada week ini akan disertakan file pendukung berupa file text.

Buatlah sebuah program yang digunakan untuk menampilkan, mencari, dan menambahkan data (dalam .txt yang disediakan). Program akan memiliki 4 menu utama.

Contoh tampilan menu utama adalah sebagai berikut:



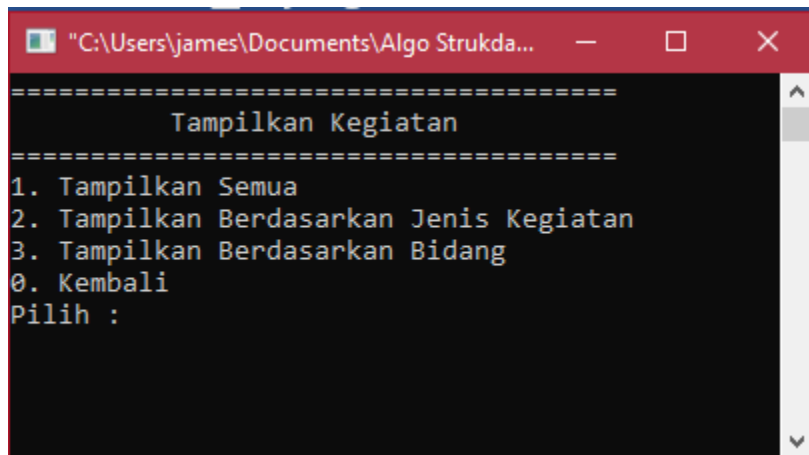
```
=====
Daftar Kegiatan di UMN
=====
1. Tampilkan Kegiatan
2. Urutkan Kegiatan
3. Cari Kegiatan
4. Tambah Kegiatan
0. Keluar
Pilih :
```

Menu 1 : Tampilkan Kegiatan

Pada menu ini, kalian akan diminta untuk membuat 3 bentuk tampilan, yakni:

- Menampilkan semua data.
- Menampilkan semua data berdasarkan jenis yang di-input.
- Menampilkan semua data berdasarkan bidang yang di-input.

Contoh bentuk tampilan adalah sebagai berikut :



```
=====
Tampilkan Kegiatan
=====
1. Tampilkan Semua
2. Tampilkan Berdasarkan Jenis Kegiatan
3. Tampilkan Berdasarkan Bidang
0. Kembali
Pilih :
```

Contoh tampilan pertama:

```

=====
Daftar Semua Kegiatan
=====
| ID | Nama | Bidang | Jenis |
=====
| 055 | AIESEC UMN | Sains & Sosial | UKM |
| 072 | Aikido | Olahraga | Komunitas |
| 004 | Capoeira | Olahraga | UKM |
| 078 | Floor Ball | Olahraga | Komunitas |
| 053 | Game Development Club (GDC) | Sains & Sosial | UKM |
| 073 | Ikatan Bikers UMN (IBU) | Unspecified | Komunitas |
| 024 | J-Cafe | Seni & Budaya | UKM |
| 077 | Kokonat | Unspecified | Komunitas |
=====

| 023 | Tracce | Seni & Budaya | UKM |
| 058 | U-Store | Sains & Sosial | LSO |
| 021 | Ultima Sonora | Seni & Budaya | UKM |
| 029 | Ultima Toys | Seni & Budaya | UKM |
| 005 | Ultimatum Taekwondo | Olahraga | UKM |
| 075 | UMN Board Game | Unspecified | Komunitas |
| 054 | UMN English Student Council (UESC) | Sains & Sosial | UKM |
| 057 | UMN Medical Center | Sains & Sosial | LSO |
| 056 | UMN Programming Club (UMNPC) | Sains & Sosial | UKM |
| 006 | UMN Softball Team | Olahraga | UKM |
| 028 | UMN Symphony Orchestra (USO) | Seni & Budaya | UKM |
| 009 | Voli | Olahraga | UKM |
=====
Tekan apapun untuk lanjut. . .

```

Menu 2 : Urutkan Kegiatan

Pada menu ini, program akan mengurutkan data berdasarkan nama. Program akan menampilkan pesan jika *sorting* berhasil dilakukan. *Sorting* yang ada di menu ada 3, yaitu *bubble sort*, *selection sort*, dan *merge sort*. *Bubble sort* dan *selection sort* bersifat wajib, sedangkan *merge sort* bersifat optional.

Contoh tampilan menu 2 beserta pesan yang ditampilkan:

```

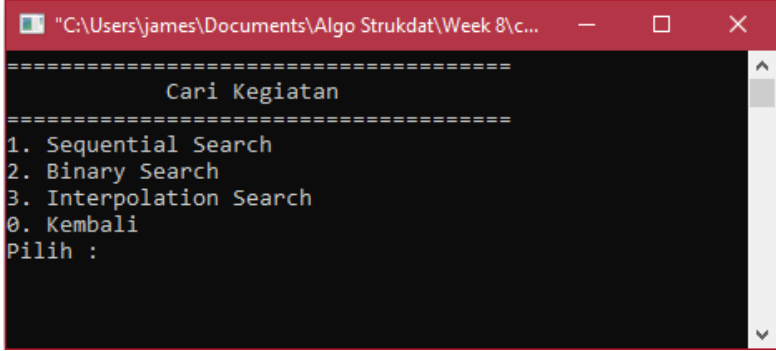
=====
Urutkan Kegiatan
=====
1. Bubble Sort
2. Selection Sort
3. Merge Sort
0. Kembali
Pilih : 1
Data berhasil di sort!
Tekan apapun untuk lanjut. . .

```

Menu 3 : Cari Kegiatan

Pada menu ini, program akan melakukan pencarian menggunakan algoritma *sequential search* (wajib), *binary search* (wajib), dan *interpolation search* (optional).

Contoh tampilan menu 3:

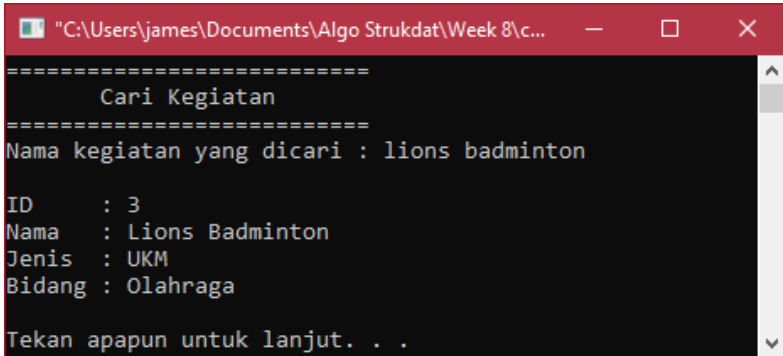


```

=====
                  Cari Kegiatan
=====
1. Sequential Search
2. Binary Search
3. Interpolation Search
0. Kembali
Pilih :
  
```

Ketika memasuki salah satu menu, user akan diminta *keyword*. *Keyword* berupa **nama** dari kegiatan yang akan dicari. *Keyword* merupakan *case insensitive*.

Contoh tampilan jika kegiatan ditemukan:



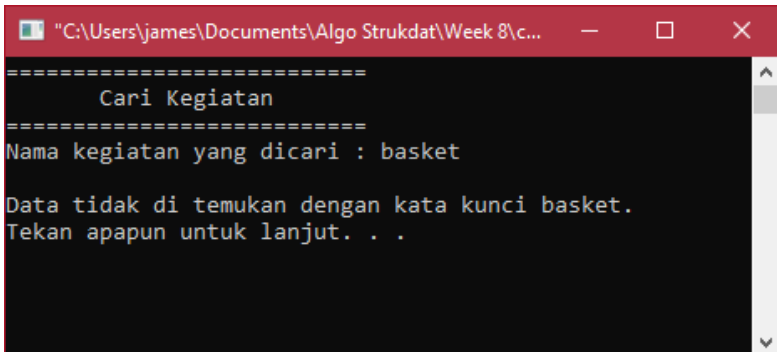
```

=====
                  Cari Kegiatan
=====
Nama kegiatan yang dicari : lions badminton

ID      : 3
Nama    : Lions Badminton
Jenis   : UKM
Bidang  : Olahraga

Tekan apapun untuk lanjut. . .
  
```

Contoh tampilan jika kegiatan tidak ditemukan:



```

=====
                  Cari Kegiatan
=====
Nama kegiatan yang dicari : basket

Data tidak di temukan dengan kata kunci basket.
Tekan apapun untuk lanjut. . .
  
```

Menu 4 : Tambah Kegiatan

Pada menu ini, user dapat menambahkan kegiatan baru ke dalam file .txt yang tersedia.

Contoh tampilan untuk tambah kegiatan:

```

=====
                    Tambah Kegiatan
=====
Keterangan :
Jenis :                      Bidang :
- UKM                       - Olahraga
- Komunitas                 - Sains & Sosial
- LSO                       - Seni & Budaya
                          - Unspecified

Nama Kegiatan : traditional dance
Jenis          : asd
Warning! Input tidak valid!
Jenis          : komunitas
Bidang         : seni
ID             : 67
Data berhasil ditambahkan!
Tekan apapun untuk lanjut. . .

```

Garis bawah merah merupakan input dari user, sedangkan garis bawah biru merupakan random number yang **tidak boleh** sama dengan ID yang sudah ada.

Note: gunakan library time.h agar mendapatkan hasil random number yang optimal.

Format penamaan: [NIM]_[Nama]_Challenge02.c

REFERENSI